Function values, application, and currying

Status: first slice implemented (D177); the full design is not. The block-scoped form `logic name params = body.` is implemented for values and functions. The definition scopes over the sentences that follow; a function definition parses to `Let(name, Lambda(params, body), rest)`, and a call to the bound name lowers to a closure call. The `function` keyword, `close`-terminated bodies, `in`, automatic currying, recursion markers, and package classification remain design.

Decided

1. The surface keyword is `logic`; the first slice reuses the existing
2. `Let`/`Lambda` nodes rather than renaming the core IR.
3. Named functions use the definition form `logic f x = body.` with parameters
4. after the name; the definition scopes over the remaining sentences.
5. Anonymous functions use `function x -> ...`; this is still design.
6. The full design's body terminator is `close`; the first slice uses a sentence
7. period and block scope. Indentation may replace `close` later.
8. Typed currying is the target from the start, not a dynamic-only phase.
9. `PartialApplication` is the call with missing arguments;
10. `ArgumentHole` is an omitted **call argument**. `ParameterHole` is reserved
11. for a future function-declaration parameter hole and is not implemented now.
12. Holes are call-argument only for the first slice.
13. Multiple dispatch selects behavior; currying and holes shape function values.

Function values

A function value carries:

1. a parameter signature: arity, parameter types, result type;
2. an effect signature;
3. its closure body and captured environment;
4. the number of leading arguments already supplied.

Application is left-associative. Supplying fewer arguments than the arity returns a new function value with the supplied arguments evaluated once and the remaining signature. Supplying the complete arity runs the body. Supplying more arguments applies the result if it is callable.

The **implementation** is nested unary closures. The **authority** is the signature table: arity, types, effects, and dispatch. Currying never guesses a tuple or a missing argument.

`f 1` where `f` has arity two returns a function awaiting the second argument. `f(1 _)` is an explicit partial application; it fills the declared call argument position and preserves the remaining signature. Automatic currying handles leading arguments; `ArgumentHole` handles non-leading omissions.

`logic` and `function`

```
logic add x y = add(x y) close
logic inc x = add(x 1) close
logic twice f x = f(f(x)) close

function x -> add(x 1) close
```

`logic name params = body close` is the full design: it binds a named function, and `function params -> body close` is an anonymous function value. `logic name = expression close` binds a non-function value; when the expression is a function, the result is callable.

The first slice (D177) uses a period and block scope instead of `close`:

```
logic inc x = add(x 1).
logic add2 x y = add(x y).
add2(inc(2) 3).
```

The definition scopes over the sentences that follow; the body is one expression. A call to the bound name lowers to a closure call. `in`, `close`, `function`, automatic currying, and recursion are not part of this slice.

The body terminator is currently `close`. Nested `match` also uses `close`; the parser must balance the block. Indentation can replace `close` later without changing the signature/closure model.

Recursion

Scale definitions use an explicit definition-site word:

1. `logic f x = ...` — ordinary; `f` is not in scope in its own body.
2. `cycle f x = ...` — self-recursive; `f` is in scope.
3. `cycle f x = ... and g y = ...` — mutual recursion; every name is in scope in every body.

The plain name is the recursive reference. There is no `recurse` word. A `logic` body that calls its own name is a named refusal.

Recursion is not one global default. The definition-site marker is the primary Scale mechanism. The package boundary has a separate classification path.

F# package classification

When Scale calls into an F# package over IPC, the recursion kind must come from the package's F# semantics, not from an "always recursive" rule. Both tiers emit the same classification record; the tier is a property of the package, not of the record.

Tier A — automatic, default. Read the F# compiler front end directly. Every binding is already classified as `let`, `let rec`, or `let rec … and …`; that classification is the package's actual semantics. Reading it off is a translation, not an inference. No manifest is needed; most packages use only this path.

Tier B — explicit, opt-in. A package manifest declares the recursion kind per binding, with an optional reason. This is for packages whose initialization graph is cross-module or otherwise not visible to F# read-off. The manifest is version-controlled with the package, so every override is auditable.

The explicit tier still runs F# read-off and compares. Explicit declarations do not replace FCS analysis; they override its classification. Every declaration is therefore one of:

1. agrees with FCS (silent);
2. disagrees with FCS (diagnostic);
3. disagrees with FCS and carries a declared reason (accepted and tracked).

Overriding a disagreement without a reason is a build error. This makes the explicit path a checked claim, not an unchecked assertion. F# diagnostics (FS0040, unused recursive, value restriction) remain in force.

The two tiers converge at one **classification record** consumed by Scale's definition words. The record has no "source tier" field. The explicit path plugs in at the "classify each binding" step: it reads the manifest and accepts or overrides FCS; everything downstream is unchanged. The Scale word's definition-site marker is the emitted form of the record, so the three levels line up per binding: package manifest (boundary), Scale marker (definition site), FCS (arbiter). A disagreement between any pair is a diagnostic, never silent divergence.

Package modes:

1. **pure automatic** — no manifest; FCS only; most packages;
2. **mixed** — FCS with per-binding overrides;
3. **strict explicit** — every binding declared; a missing declaration fails.

All three emit the same Scale markers and classification record. The mode is a per-package annotation, not a language fork.

The explicit path has a second implementation cost: a regression suite that asserts every declared binding agrees with FCS or carries a declared reason. That cross-tier consistency test is the contract.

Value recursion is a separate initialization phase: the image builder rejects a recursive value whose dependency is not initialized. Function recursion is the definition-site marker above; value and function recursion are not conflated.

Multiple results

Scale words may pop and push several values; F# functions and Elixir functions return one term. The compatibility rule is:

1. A function value is **single-result**. Currying a multi-result word is a named refusal.
2. Multi-result words remain concatenative stack effects and cannot be used as function values.
3. If F# code needs multiple results, use an explicit tuple/record type later, not an implicit tuple from the stack.

This keeps F# and Elixir call semantics honest and avoids guessing which stack values form the result tuple.

Signatures and dispatch

Every function has a signature record in the Prelude/image:

```
Signature = parameters: Type list, result: Type, effects: EffectSet
```

A partial application checks arity and effects against the remaining signature. A complete call dispatches. Multiple dispatch selects the method from the actual argument types; a partial value carries the unresolved signature and does not commit to a method until it is complete. A disagreement between the signature table and a method is a named diagnostic.

Parameter type annotations and inference are part of P4. The surface may allow `logic f (x: T) (y: T) : R = ... close`; the signature table is still the authority. Until P4 supplies inference, unannotated parameters are inferred from the body's word signatures or refused.

Open choices

1. Recursion delivery order: land `logic recursive` first, then mutual `and`,
2. Recursion delivery order: land `logic` first, then `cycle`, then mutual `and`; or land all three together.
3. Type syntax: inference from word signatures is chosen; decide later whether explicit `(x: T)` and `: R` annotations are also accepted.
4. `close` conflict: use the same `close` for match and function bodies, or add a distinct function terminator while keeping `close` for matches?
5. Package classification record: freeze its schema and manifest syntax when the F# package interop lane starts.

Multiple results are settled: functions are single-result; multi-result words remain stack effects and cannot be curried.